

Real-Time Chat Application — Engineering Deep Dive

Assumptions Validation, Technical Trade-offs, Metrics, Problem Solving & Key Features

Q1: How did you validate your technical assumptions during development?

Answer:

- **WebSocket Latency vs HTTP Overhead:** Assumed HTTP polling would waste bandwidth. *Validation:* Measured packet payloads—WebSockets achieved sub-15ms message delivery with ~2-byte frame overhead vs ~800-byte HTTP header overhead per polling request.
- **DB Connection Pool Safety:** Assumed standard FastAPI route dependency injection (`get_db`) would exhaust connection pools during long-lived WebSockets. *Validation:* Simulated concurrent connections and observed pool starvation; validated that wrapping queries in short-lived context managers (with `SessionLocal()` as `db`) immediately released connection slots.
- **Query Parameter JWT Auth:** Assumed passing JWTs in URL query parameters (`ws://.../ws?token=JWT`) would securely authenticate clients despite browser header limitations. *Validation:* Verified that expired/forged tokens were rejected before socket upgrades (`websocket.accept()`).

Q2: What key architectural trade-offs did you make, and why?

Answer:

- **SQLite (Dev) vs MySQL (Production):** Chose SQLite for local development and zero-configuration `pytest` runs, trading off high multi-writer concurrency for developer velocity. Configured SQLAlchemy ORM to switch seamlessly to MySQL in Docker Compose for production-grade row-level locking.
- **In-Memory ConnectionManager vs Early Redis Pub/Sub:** Built an in-memory dictionary-based manager (`user_id -> WebSocket`) to keep the codebase clean and lightweight for single-node deployments, while documenting Redis Pub/Sub patterns for multi-node horizontal scaling.
- **Vanilla JS/CSS vs Frontend Framework (React/Vue):** Used raw HTML5, CSS3 glassmorphism, and native JavaScript to eliminate build tools (Vite/Webpack) and npm dependencies, trading off declarative state management for lightweight, instant browser execution.

Q3: How did you measure success, performance, and impact for this project?

Answer:

- **Real-Time Latency:** End-to-end message broadcast times between active WebSocket clients remained under 15ms across room channels and direct DMs.
- **Connection Pool Stability:** Zero DB pool exhaustion or connection leaks during multi-hour connected WebSocket sessions due to context-managed database sessions.
- **Automated Test Reliability:** Achieved 100% pass rate across unit and integration tests using `pytest` with an isolated SQLite database fixture (`test_chat.db`).
- **Clean Architecture Standards:** Successfully decoupled backend layers into models, validation schemas, auth services, REST routes, and WebSocket brokers, making the project maintainable and easy to review.

Q4: What major problems did you face while building it, and how did you solve them?

Answer:

- **Problem 1: Connection Pool Exhaustion in WebSockets:** Persistent socket handler loops held onto injected DB sessions, starving the pool. *Solution:* Refactored to short-lived `SessionLocal()` context managers scoped strictly to individual queries.
- **Problem 2: Browser Header Restrictions on WebSocket Handshakes:** Browser new `WebSocket()` APIs forbid custom headers like `Authorization: Bearer`. *Solution:* Authenticated clients via query tokens passed during handshakes and verified signatures before socket acceptance.
- **Problem 3: Typing Event Spam & Ghost Users:** Keystrokes spammed socket frames and abrupt disconnects left ghost online users. *Solution:* Implemented 2-second JavaScript debouncing on typing inputs and added a server lifespan shutdown hook to bulk-reset `is_online = False` in MySQL.

Q5: What key features are you most proud of in this application?

Answer:

- **Unified Single-Table Message Model:** Flexible schema supporting both public chat rooms (`room_id`) and private direct messages (`recipient_id`) within a single database table.
- **Full-Duplex Typing Indicators & Online Tracking:** Dynamic client state synchronization updating user presence and typing bubbles in real time without manual page refreshes.
- **Glassmorphic Dark-Mode UI & Message Search:** Premium responsive UI with real-time text query search across message histories.
- **Production-Ready Docker Setup:** Clean multi-container `docker-compose.yml` orchestration with MySQL health checks and Redis caching.